

Predicting Smartphone Addiction with Logistic Regression

Carlos Manuel Martínez Vázquez
Instituto Tecnológico y de Estudios Superiores de Monterrey

September 13, 2026

Abstract

This work presents the implementation of a logistic regression model for a binary classification problem, whose purpose is to predict smartphone addiction from usage habits. The dataset comes from Kaggle’s *Playground Series S6E8* competition and contains 691,369 training records with 4% to 19% missing values across every column. Two implementations are compared: one written from scratch in NumPy and one on scikit-learn. The model is evaluated on a held-out validation set and on the competition’s test set, and its bias and variance are diagnosed from the training and validation cost. Adding polynomial features and a small L2 penalty raised the test AUC from 0.913 to 0.925.

1 Introduction

Excessive smartphone use has become a public health concern associated with sleep disorders, anxiety and poor academic performance. Identifying people with problematic usage patterns early would allow preventive interventions to be designed.

This work approaches the problem as a binary classification task: given nine numeric variables describing usage habits and three categorical context variables, the goal is to predict the probability that a person exhibits smartphone addiction.

Since the dataset contains missing values in every one of its columns, this first deliverable centres on the data preparation stage, paying particular attention to statistically justifying the imputation strategy rather than assuming a default criterion. The model itself was implemented directly in *NumPy*, which keeps every step of the algorithm explicit and allows the gradient to be verified numerically.

2 ETL

2.1 Data source

The data comes from the *Predicting Smartphone Addiction* competition (Playground Series, Season 6, Episode

8) on Kaggle. The dataset is distributed already split into two files:

- **train.csv**: 691,369 records and 14 columns.
- **test.csv**: 296,302 records and 13 columns.

The column missing from **test.csv** is the target variable **addicted_label**, since the test labels are withheld by the platform in order to score submitted predictions. **test.csv** therefore serves as the test set, and a validation set is carved out of **train.csv** (Section 4.1).

The target variable is binary and moderately imbalanced: 490,474 positive cases (70.94%) against 200,895 negative ones (29.06%). No duplicate records were found.

2.2 Variable description

Table 1 describes the available variables.

Variable	Type
age	Numeric
daily_screen_time_hours	Numeric
social_media_hours	Numeric
gaming_hours	Numeric
work_study_hours	Numeric
sleep_hours	Numeric
notifications_per_day	Numeric
app_opens_per_day	Numeric
weekend_screen_time	Numeric
gender	Categorical
stress_level	Categorical
academic_work_impact	Categorical
addicted_label	Target (binary)

Table 1: Dataset variables.

2.3 Exploratory analysis

The correlation matrix (Figure 1) reveals two relevant findings.

First, the variables most correlated with the target are **daily_screen_time_hours**

(0.61), `weekend_screen_time` (0.59) and `social_media_hours` (0.53). After these three there is a sharp drop: `work_study_hours` (0.25) and `gaming_hours` (0.21), while `age`, `sleep_hours`, `notifications_per_day` and `app_opens_per_day` show correlations of 0.06 or lower.

Second, those same three screen-time variables are strongly correlated *with each other*, the highest being between `daily_screen_time_hours` and `weekend_screen_time` (0.80).

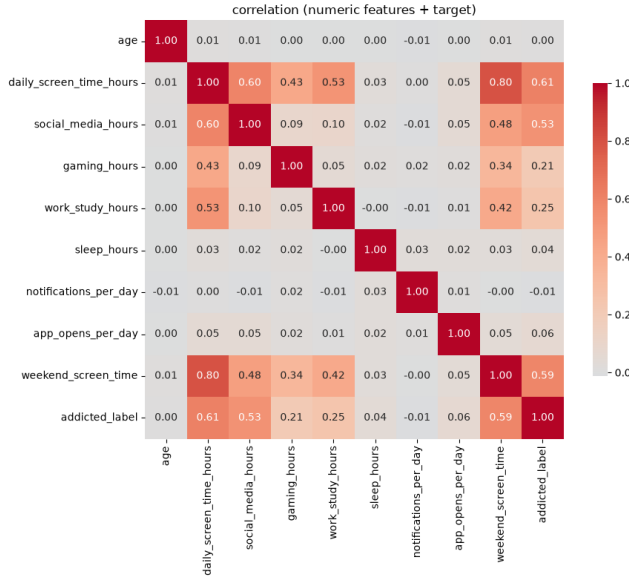


Figure 1: Correlation matrix of the numeric variables and the target.

2.4 Missing values

Every column in the training set contains missing values, in proportions ranging from 4.18% to 19.38% (Table 2). This is the main data quality problem of the project.

Variable	% missing
<code>social_media_hours</code>	19.38
<code>gaming_hours</code>	18.34
<code>weekend_screen_time</code>	16.21
<code>daily_screen_time_hours</code>	13.86
<code>app_opens_per_day</code>	11.67
<code>notifications_per_day</code>	9.78
<code>stress_level</code>	7.98
<code>work_study_hours</code>	7.45
<code>sleep_hours</code>	6.43
<code>academic_work_impact</code>	6.40
<code>gender</code>	4.20
<code>age</code>	4.18

Table 2: Percentage of missing values per variable.

Simple imputation was chosen to fill these values. The choice of the statistic used for the imputation is based on the skewness test presented in the following section.

2.5 Skewness test and choice of the median

The choice between mean and median as the imputation statistic depends on the symmetry of the distribution. The mean is sensitive to outliers, since it is dragged toward the long tail of a skewed distribution, whereas the median is robust to them. The skewness coefficient of each numeric variable was therefore computed (Table 3), using the conventional threshold of $|g| > 1$ to consider a distribution strongly skewed.

Variable	Skewness
<code>work_study_hours</code>	0.56
<code>gaming_hours</code>	0.54
<code>social_media_hours</code>	0.47
<code>notifications_per_day</code>	-0.21
<code>daily_screen_time_hours</code>	-0.13
<code>app_opens_per_day</code>	-0.13
<code>weekend_screen_time</code>	-0.06
<code>age</code>	-0.04
<code>sleep_hours</code>	-0.01

Table 3: Skewness coefficient per numeric variable.

No variable exceeds an absolute skewness of 0.56, so all distributions can be considered approximately symmetric. Figure 2 illustrates the effect of skewness on the separation between mean and median: even at a skewness of 1.0 (higher than that of any variable in this dataset) the difference between the two statistics is only 0.15 standard deviations, and it vanishes in the symmetric case.

The **median** was selected for three reasons:

1. In symmetric distributions the median coincides with the mean, so using it costs nothing; in exchange, it offers protection should the symmetry not hold.
2. The imputed value is replicated across up to 133,995 records (`social_media_hours`). The median is a value actually observed in the data, whereas the mean may not be: in `notifications_per_day`, an integer-valued variable, the mean is 145.89 notifications, a quantity no real record can hold.
3. It matches the behaviour of standard imputation utilities, which keeps later iterations of this work directly comparable.

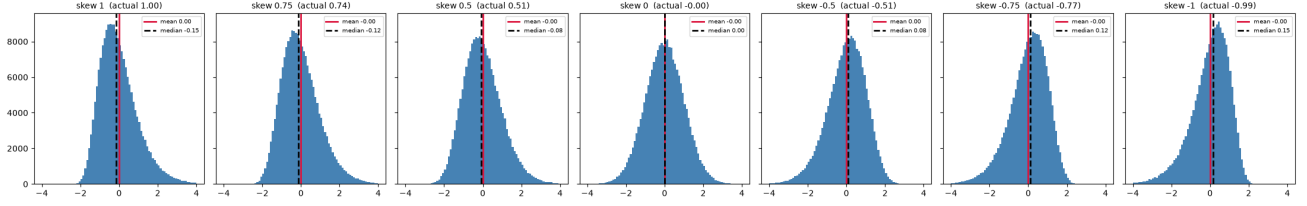


Figure 2: Simulated distributions with controlled skewness, standardized to mean zero and standard deviation one. The red line marks the mean and the dashed black line the median. As skewness increases, the mean shifts toward the long tail while the median stays in place.

For the three categorical variables the mode was used, since the median is meaningless for these types of variables, as they classify rather than being numerical values.

2.6 Scaling and clipping of extreme values

The numeric variables were standardized using the z -score:

$$z = \frac{x - \mu}{\sigma} \quad (1)$$

This step is essential because the variables are on heterogeneous scales: `notifications_per_day` has a standard deviation of 65.92 against 0.93 for `gaming_hours`, a difference of roughly 70 times. Without standardization, principal component analysis would be dominated by the higher-magnitude variables purely because of their units of measurement.

Values were then clipped at ± 3 standard deviations. This criterion was chosen over the interquartile range (IQR) based on the result of the skewness test: the $\pm 3\sigma$ rule assumes an approximately symmetric distribution, a condition satisfied by this dataset, where the maximum skewness is 0.56. The IQR criterion ($Q_1 - 1.5 \text{ IQR}$, $Q_3 + 1.5 \text{ IQR}$) is preferable for strongly skewed distributions, since it makes no symmetry assumption; with no relevant skewness present, it would offer no advantage over the simpler criterion.

Clipping modified only 0.066% of the values, roughly one in every 1,500. This confirms that the distributions were already naturally bounded, so the operation acts as a low-cost preventive measure rather than a substantive correction.

2.7 Encoding of categorical variables

The three categorical variables were one-hot encoded, dropping the first category of each.

2.8 Principal component analysis

PCA was implemented manually through the eigendecomposition of the covariance matrix of the standardized data, using `numpy.linalg.eigh`, which is appropriate for symmetric matrices. The components were sorted from highest to lowest explained variance.

	Explained variance	Cumulative variance
PC1	0.2747	0.2747
PC2	0.1160	0.3907
PC3	0.1114	0.5021
PC4	0.1100	0.6122
PC5	0.1080	0.7202
PC6	0.1055	0.8257
PC7	0.0998	0.9255
PC8	0.0485	0.9740
PC9	0.0260	1.0000

Table 4: Variance explained by each principal component and cumulative variance.

Table 4 and Figure 3 show the variance captured by each component. Only PC1 stands out, explaining 27.47% of the variance, roughly two and a half times what a single variable would contribute; its highest loadings correspond to `daily_screen_time_hours` (-0.58), `weekend_screen_time` (-0.54) and `social_media_hours` (-0.39). Squaring each loading gives that variable’s share of the component, so these three account for $0.58^2 + 0.54^2 + 0.39^2 \approx 80\%$ of PC1, with the remainder spread thinly across the other six variables. PC1 therefore corresponds to the correlated block identified in the exploratory analysis, collapsed into a single axis of overall screen usage.

The first seven components were retained, preserving 92.55% of the total variance. The final input vector to the model consists of twelve columns: seven principal components and five binary variables.

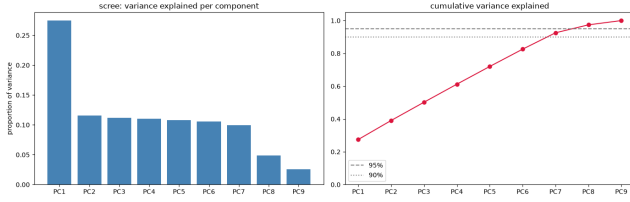


Figure 3: Variance explained per component and cumulative variance.

3 Logistic Regression

3.1 Sigmoid function

The sigmoid function transforms the linear combination of features and parameters into a probability within the interval $(0, 1)$:

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}} \quad (2)$$

In the implementation, the argument of the exponential is clipped to the interval $[-500, 500]$ to avoid numerical overflow of the exponential function for very negative values.

3.2 Cost function

Binary cross-entropy was used:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y_i \log(h_i) + (1 - y_i) \log(1 - h_i)] \quad (3)$$

Probabilities are clipped to the interval $[10^{-10}, 1 - 10^{-10}]$ before evaluating the logarithm, thereby avoiding the $\log(0)$ indeterminacy.

3.3 Gradient descent

The parameters are updated iteratively through:

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_i - y_i) x_{ij} \quad (4)$$

where α is the learning rate, m the number of samples, h_i the prediction for sample i and y_i its true label.

A column of ones was appended to the feature matrix, so that θ_0 acts as the intercept term. The parameters were initialized at zero, except for the intercept, which was assigned the log-odds of the base rate:

$$\theta_0 = \log \frac{\bar{y}}{1 - \bar{y}} = 0.8916 \quad (5)$$

In this way the model starts out correctly predicting the average proportion of the positive class, and gradient descent only has to learn the deviations from that baseline.

3.4 Gradient checking

To verify that the analytical expression of the gradient was implemented correctly, it was compared against a numerical approximation by central differences:

$$\frac{\partial J}{\partial \theta_j} \approx \frac{J(\theta_j + \epsilon) - J(\theta_j - \epsilon)}{2\epsilon} \quad (6)$$

With $\epsilon = 10^{-5}$ on the parameter corresponding to PC1, the numerical gradient was 2.1495306×10^{-7} and the analytical one 2.1495081×10^{-7} , agreeing to six significant figures. This test confirms the correctness of the implementation independently of whether the cost decreases during training.

3.5 Training process

The model was trained on the training split (553,095 records and 12 features; see Section 4.1) with a learning rate of $\alpha = 0.5$. The stopping criterion was set to when the parameters stop changing appreciably between consecutive iterations (absolute tolerance of 10^{-6}).

It is worth noting that an exact equality comparison between parameter vectors is never satisfied when working with floating-point numbers, since gradient descent keeps producing changes in the last decimal places indefinitely; a tolerance-based comparison was therefore used.

Training stopped at epoch 1,663. The cost decreased from 0.60286 to 0.34669, as shown in Figure 4. The initial value corresponds exactly to the entropy of a 71/29 distribution, which verifies the correct initialization of the intercept: with every weight at zero the model predicts the base rate for every record, so it starts precisely on the baseline. The validation cost, computed with the same parameters at every epoch, is plotted alongside.

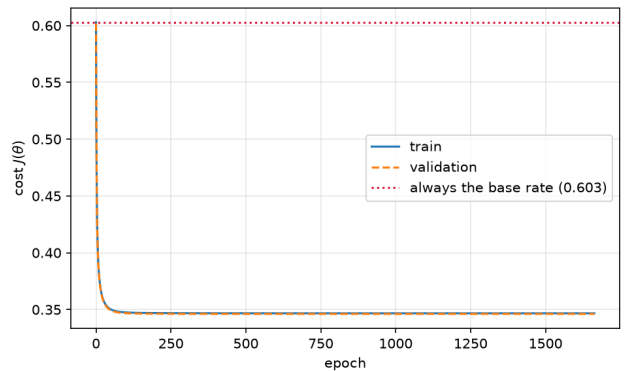


Figure 4: Training and validation cost per epoch. The dotted line is the cost of always predicting the base rate.

3.6 Results

The evaluation metric of the competition is the area under the ROC curve (AUC), computed on the predicted probabilities. Since AUC depends only on the ordering of the predictions and not on a decision threshold, there was no need to tune such a threshold or to correct the class imbalance.

AUC was implemented manually from its equivalence with the Mann-Whitney U statistic:

$$\text{AUC} = \frac{R_+ - \frac{n_+(n_++1)}{2}}{n_+ \cdot n_-} \quad (7)$$

where R_+ is the sum of the ranks of the positive observations, and n_+ and n_- the number of positive and negative observations respectively.

Metric	Value
AUC (training)	0.91096
AUC (validation)	0.91106
AUC (test, Kaggle)	0.91342
Final cost $J(\theta)$, training	0.34669
Final cost $J(\theta)$, validation	0.34615
Baseline cost	0.60286
Epochs to convergence	1,663

Table 5: Results of the from-scratch logistic regression.

The model achieved an **AUC of 0.911** on training and validation and 0.913 on the competition test set (Table 5). As a validity check on the metric implementation, AUC was evaluated on the same predictions in random order, yielding 0.5007, a value consistent with what is expected for a random classifier.

The final cost of 0.34669 represents a reduction of roughly 43% against the cost of 0.60286 corresponding to a model that always predicts the base rate. It is important to note that this value is not an error rate: cross-entropy is a measure of confidence and is not bounded above. Its direct interpretation is that the model assigns on average a probability of $e^{-0.34669} = 0.707$ to the correct class.

The cost cannot approach zero because the distributions of the two classes overlap considerably, as observed in the exploratory analysis: there are records with identical screen usage levels in both classes, so a portion of the error is irreducible.

4 Model Evaluation

4.1 Validation and test sets

The competition’s `test.csv` (296,302 records) carries no labels, so it cannot be scored locally; it is nevertheless a genuine test set, since submitting predictions returns an AUC computed on records the model has

never seen. The validation set therefore has to come out of `train.csv`, which was split by position: the first 80% (553,095 records) for training and the last 20% (138,274) for validation. The rows arrive already shuffled, and the positive rate is 0.7092 in the first part against 0.7103 in the second, so no stratification was needed.

The split is performed before any preprocessing. The imputation medians, the z-score parameters and the PCA are all quantities learned from the data; fitting them on the complete file (train/val) would let the validation rows influence the transformation applied to the training rows, inflating the validation score without improving the model.

Table 6 shows that validation predicted the test result to within 0.003 for all three submissions, which is the evidence that the split is honest.

Model	Train	Val	Test
Linear (scratch)	0.91096	0.91106	0.91342
Linear (framework)	0.91063	0.91073	0.91312
Poly2 + L2 (framework)	0.92229	0.92243	0.92484

Table 6: AUC on the three sets. Test is the Kaggle private leaderboard score (Figure 5).

4.2 Bias and variance

Both of these can be read from Figure 4 without fitting any other model. The gap between the two curves within the image measures variance; the level at which they settle, relative to the base-rate baseline, measures bias.

Variance is low. The validation curve lies on top of the training curve for the entire run and never turns upward. At convergence the training cost is 0.34669 and the validation cost 0.34615; a model with meaningful variance would show the training cost fall continuously while the validation cost bottoms and then climbs.

Bias is moderate. Both curves flatten at 0.347 and stop. That is 43% below our initial base rate of 0.603, so the features carry real signal; but the cost is far from zero, and more tellingly the training cost itself stops falling: the model cannot fit even the data it was trained on any better than this.

4.3 Level of fit and regularization

The model is **mildly underfitting**. Training and validation cost are essentially almost equal, which rules out overfitting, and both stop improving well above zero, which rules out that it is a good fit. The productive direction is therefore more capacity, not more data: the curves are already flat, so more rows would not move them.

Submission and Description		Private Score	Public Score	Selected
	submission_framework.csv Complete (after deadline) · 2d ago · poly2 reg 0.001	0.92484	0.92404	☐
	submission_framework.csv Complete (after deadline) · 5d ago	0.91312	0.91277	☐
	submission_scratch.csv Complete · 14d ago · First iteration logistic regression.	0.91342	0.91310	☐

Figure 5: Kaggle scores on `test.csv`, newest first: the polynomial framework model (0.92484), the linear framework model (0.91312) and the from-scratch linear model (0.91342). The private score is computed on the full `test.csv` and is the one reported; the public score uses a subset of it.

Regularization is a tool for the opposite problem. An L2 penalty trades a little bias for a reduction in variance, so its benefit is bounded by how much variance there is to remove within the model, and Section 4.2 found almost none. For this linear model it is therefore not really needed. It was implemented and tested regardless, adding $\frac{\lambda}{2m} \sum_{j \geq 1} \theta_j^2$ to the cost with the intercept exempt, because the next step in Section 4.5 raises the parameter count from 13 to 91, and that is where a penalty begins to earn its place.

λ	Epochs	Val AUC
0	1,663	0.91106
1,000	1,470	0.91118
10,000	767	0.91148
100,000	193	0.90822

Table 7: L2 sweep on the from-scratch model.

The result is as predicted (Table 7): the best value, $\lambda = 10^4$, improves validation AUC by only +0.0004, and larger values shrink the weights past usefulness and the score collapses.

4.4 Framework implementation

The same model was built on scikit-learn’s `SGDClassifier` with `loss="log_loss"`, which minimizes the identical cross-entropy by gradient descent; the only difference is that the library takes each step on one sample at a time instead of on the full batch. Its L2 strength `alpha` corresponds to λ/m of the scratch model, because the library averages the cost over the batch. With linear features the two agree closely: 0.91063 / 0.91073 (train / validation) against 0.91096 / 0.91106 for the scratch version.

4.5 Improving the model: polynomial features

Since the diagnosis pointed out to the capacity of the model, the framework version was given degree-2 polynomial features. From each pair of inputs x_i, x_j this adds the columns x_i^2 and $x_i x_j$, expanding the 12 features to 90 (12 originals, 12 squares, 66 interactions). The model is still logistic regression, only with more columns. The squared terms let the decision boundary curve instead of being a flat plane in PCA space; the interaction terms let the effect of one component depend on another, so that, for instance, heavy screen time can matter more when sleep is also low. PCA and the classification threshold were kept as they were.

Model	n_θ	Train	Val	Gap
Linear, $\alpha_{L2} = 10^{-4}$	13	0.91063	0.91073	+0.0001
Poly2, $\alpha_{L2} = 10^{-4}$	91	0.92125	0.92128	+0.0000
Poly2, $\alpha_{L2} = 10^{-3}$	91	0.92229	0.92243	+0.0001

Table 8: Capacity, then regularization. AUC; gap is validation minus training.

Table 8 shows the effect in two steps. Capacity alone moved validation AUC by +0.0106; tuning the L2 strength on top of it added a further +0.0012.

The decisive detail is the last column. Going from 13 to 91 parameters is the change that should introduce overfitting if the model were near that regime; instead the train/validation gap never exceeded +0.0001, and validation remained fractionally ahead of training throughout. The model in fact got better at the problem, not better at the training set. The same experiment on the scratch model, adding only the squared components, reached a validation AUC of 0.92135. On the competition test set the framework model’s improvement carried over: 0.91312 to 0.92484, a gain of +0.0117 that matches the validation gain of +0.0117.

4.6 Confusion matrix

Figure 6 shows both models on the validation set at the default threshold of 0.5 (a record is called “addicted” when $h(x) \geq 0.5$). The threshold was not tuned.

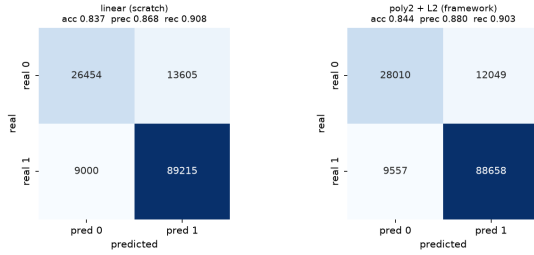


Figure 6: Confusion matrices on the validation set, threshold 0.5.

For the linear model the errors are lopsided: 13,605 false positives against 9,000 false negatives. It catches 90.8% of the addicted records but only 66.0% of the non-addicted ones. With 71% of the data positive, a 0.5 threshold sits below the base rate, so any record the features do not clearly pull down is called addicted. Accuracy (0.8365) flatters the model, since always predicting the majority class already scores 0.709; this is why the competition uses AUC, which is threshold-free.

The improved model reduces total mistakes from 22,605 to 21,606. The gain lands almost entirely on the weak class: 1,556 more non-addicted records recovered, lifting their detection from 66.0% to 69.9%, at the cost of 557 addicted records now missed. Precision rises from 0.8677 to 0.8804 and recall falls slightly from 0.9084 to 0.9027: the model became more selective.

4.7 ROC curve

Figure 7 plots the true positive rate against the false positive rate as the threshold goes from 1 to 0. The area under the linear model’s curve, computed numerically over the swept thresholds, is 0.91101, matching the rank based value of 0.91106 from the Mann Whitney formula, so two definitions of AUC agree. The improved model’s curve sits above the linear one everywhere, so it is better at every threshold, not only at 0.5.

5 Conclusions

A logistic regression model was implemented from scratch and on scikit-learn, evaluated on a held-out validation set and on the competition’s test set. The linear model reached a test AUC of 0.913; diagnosing it as low variance and moderately biased pointed to capacity as the missing ingredient, and polynomial features with a small L2 penalty raised the test AUC to 0.925 with the train-validation gap unchanged at zero.

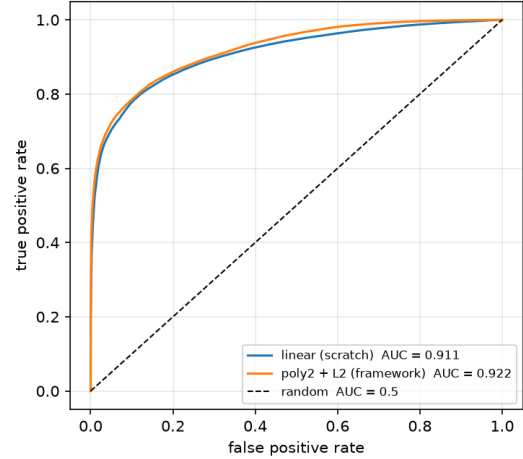


Figure 7: ROC curves on the validation set.

The treatment of missing values was the central aspect of this work. The skewness test provided a quantitative justification for using the median instead of the mean, as well as for preferring the ± 3 standard deviation criterion over the interquartile range when clipping extreme values. In both cases, the decision rested on a measurement rather than on a convention.

Principal component analysis showed that the dataset admits no relevant dimensionality reduction: seven of nine components were needed to retain 92.55% of the variance. This result is a finding in itself, since it shows that the variables are mostly independent, with the exception of the correlated screen-time block.

The most informative result was the bias-variance diagnosis itself. Read from a single plot of training against validation cost, it correctly predicted that regularization would help little (it added +0.0004) and that capacity would help a great deal (it added +0.011), and both predictions held without any overfitting appearing.

References

- [1] Kaggle. *Playground Series S6E8: Predicting Smartphone Addiction*. <https://www.kaggle.com/competitions/playground-series-s6e8>
- [2] A. Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3rd ed., O’Reilly Media, 2022.
- [3] GeeksforGeeks. *ML / Handling Missing Values*. <https://www.geeksforgeeks.org/machine-learning/ml-handling-missing-values/>
- [4] GeeksforGeeks. *Understanding Logistic Regression*. <https://www.geeksforgeeks.org/machine-learning/understanding-logistic-regression/>

- [5] GeeksforGeeks. *Principal Component Analysis (PCA)*.
[https://www.geeksforgeeks.org/machine-learning/
principal-component-analysis-pca/](https://www.geeksforgeeks.org/machine-learning/principal-component-analysis-pca/)
- [6] GeeksforGeeks. *AUC-ROC Curve*. [https:
//www.geeksforgeeks.org/machine-learning/
auc-roc-curve/](https://www.geeksforgeeks.org/machine-learning/auc-roc-curve/)